

MonPoint MS

Migración de arquitectura monolítica a microservicios

Especificación de Requisitos y Alcance

Línea base funcional, decisiones de diseño, mensajería y delimitación del alcance

Documento	02 de 04 — Especificación de Requisitos y Alcance
Proyecto	MonPoint MS — reingeniería arquitectónica de MonPointV2
Equipo	Alejandro (Marquitos) · Victor · Hugo
Periodo	31 de agosto — 6 de noviembre de 2026 (10 semanas)
Versión	3.1 — sustituye JavaFX por un cliente web en Vue 3 (sección 5.11)
Estado	Alcance cerrado. No sujeto a cambio

Contenido

- 1. Objeto y estado del documento
- 2. Línea base funcional del sistema actual
- 3. Hallazgos del análisis de código
- 4. Arquitectura de servicios
- 5. Decisiones de diseño
- 6. Requisitos funcionales por servicio
- 7. Requisitos no funcionales
- 8. Delimitación del alcance
- 9. Comunicación entre servicios: eventos y saga
- 10. Trazabilidad y criterios de verificación

1. Objeto y estado del documento

Este documento establece qué se construye y, con igual importancia, qué explícitamente no se construye. Funciona como contrato de alcance. Los requisitos se derivan del comportamiento verificado en el código fuente de MonPointV2, no de su manual técnico, que contiene inexactitudes documentadas en la sección 3.

El alcance está cerrado. Toda propuesta de ampliación, sustitución o reinterpretación se escala al Product Owner y no se implementa por iniciativa individual.

1.1 Cambios de la versión 3.0

Cambio	Origen	Secciones afectadas
MongoDB como motor de persistencia único	Requisito de la asignatura	4, 5.1–5.3, 6, 7, 8
Comunicación asíncrona entre servicios	Requisito de la asignatura	4, 5.4–5.10, 6, 7, 8, 9

Decisión invertida

La versión 2.0 establecía comunicación síncrona y excluía expresamente la mensajería y el patrón saga por su costo. Ese razonamiento sigue siendo correcto como análisis de costo; lo que cambió es la restricción. El incremento de costo está cuantificado en el Documento 01 (de 195 a 227 puntos).

2. Línea base funcional del sistema actual

Inventario de funcionalidad de MonPointV2, verificada en código. Es la referencia de paridad funcional.

Módulo	Funcionalidad verificada
Autenticación	Login por correo y contraseña. BCrypt, factor 12. Migración automática de contraseñas en texto plano. Sesión persistida en archivo cifrado.
Usuarios	Alta, consulta, modificación, baja. Política de contraseña segura. Validación de correo. Roles ADMIN, VENDEDOR, OTRO.
Productos	Alta, consulta, modificación, baja lógica. Listado sin inactivos. Control de existencias.
Clientes	Alta, consulta, modificación, baja. Nombre, correo, teléfono, dirección, fecha de registro.
Proveedores	Alta, consulta, modificación, baja. RUC único, razón social, contacto, fecha de registro.
Ventas	Registro con múltiples líneas. Validación y descuento de existencias. Cálculo de total.
Historial	Listado con ordenamiento por campo y filtro por rango de fechas. Paginación en interfaz.
Gráfica	Ventas en gráfico de barras y de pastel por rango.
Configuración	Nombre, RFC, dirección, teléfono de la empresa. Encabezado de todos los PDF.
Exportación	Ticket de venta, historial, listado de clientes y de productos en PDF.
Interfaz	Temas claro y oscuro. Animaciones. Navegación lateral según rol.

3. Hallazgos del análisis de código

3.1 Discrepancias entre documentación y código

Aspecto	Manual técnico v2.0	Código real
Biblioteca PDF	iText 7	iText 9.1.0, línea de API distinta
Roles	ADMIN y EMPLEADO	ADMIN, VENDEDOR y OTRO
Patrón	Modelo-Vista-Controlador	Capas Controlador–DAO sin capa de servicio
Esquema	Incluye proveedores y configuración	El script SQL no contiene ambas tablas

3.2 Defectos que la migración corrige

Defecto	Descripción	Corrección
Tipos monetarios	Precios y totales en coma flotante; los errores se acumulan en ventas de varias líneas.	Decimal128 en persistencia y decimal exacto en el contrato
Cifrado de sesión	Cifrado sin modo explícito, que resuelve a ECB sin vector de inicialización.	Sustitución por token
Autorización cosmética	El control de acceso oculta botones; no hay verificación en servidor.	Verificación de rol en cada servicio
Relación viva en línea de venta	Un ticket antiguo reimpresso muestra el nombre actual del producto.	Copia histórica de nombre y precio
Concatenación en consulta	Cláusula de ordenamiento construida por concatenación sin validación.	Lista blanca de campos ordenables
Credenciales expuestas	Clave de cifrado y archivo de sesión versionados; credenciales publicadas.	Variables de entorno

3.3 Elementos sin uso

- **Apache POI**: dependencia declarada, nunca utilizada.
- **Tabla de facturas**: existe en el esquema sin entidad ni lógica.
- **Rol OTRO**: no participa en ninguna regla de negocio.
- **Importador.java**: fuera de la estructura de paquetes.
- **Directorio de compilación**: versionado en el repositorio.

4. Arquitectura de servicios

#	Servicio	Responsabilidad	Colecciones propias	Peso
1	ms-auth	Identidad, roles, emisión de token	usuarios	Alto
2	ms-inventario	Catálogo, existencias, reserva de stock	productos, outbox, eventos_procesados	Medio
3	ms-ventas	Ventas, saga, folio, réplica local	ventas, productos_replica, clientes_replica, contadores, outbox, eventos_procesados	Alto
4	ms-clientes	Clientes y proveedores	clientes, proveedores	Bajo
5	ms-reportes	Configuración, proyecciones, PDF, agregaciones	configuracion, ventas_proy, productos_proy, contrapartes_proy, eventos_procesados	Medio
6	ms-notificaciones	Registro de eventos y alertas	eventos, notificaciones, eventos_procesados	Mínimo

Infraestructura	Función
api-gateway	Único punto de entrada HTTP. Enrutamiento y validación local del token.
eureka-server	Registro de instancias. Lo consume el Gateway para enrutar.
rabbitmq	Broker. Exchanges por dominio, colas por consumidor, colas de mensajes fallidos.
mongodb	Instancia única en replica set. Una base lógica y un usuario por servicio.
cliente-web	Contenedor nginx que sirve la compilación de producción del cliente Vue 3.

```

Cliente Vue (SPA) --HTTP--> api-gateway --HTTP--> [servicio de dominio] --> su base logica
                        |
                        +--> eureka-server (resolucion de rutas)

[servicio de dominio] --evento--> rabbitmq --evento--> [otro servicio de dominio]

Regla: ningun servicio de dominio invoca por HTTP a otro servicio de dominio.

```

5. Decisiones de diseño

Cada decisión incluye su justificación, porque son las que un revisor externo cuestionaría.

5.1 MongoDB como motor único para los seis servicios

Se descarta la persistencia polígota (MongoDB en unos servicios y relacional en otros): duplicaría tecnologías de datos para un equipo sin experiencia previa en ninguna de las dos en este stack. El modelo documental encaja mejor que el relacional en la mayoría de los servicios:

Servicio	Encaje	Motivo
ms-ventas	Muy alto	La venta con sus líneas es un agregado: un documento con arreglo embebido. Desaparece la tabla de detalle.
ms-reportes	Muy alto	Sus proyecciones son documentos desnormalizados listos para consultar.
ms-notificaciones	Muy alto	Registro de eventos de estructura variable.
ms-auth · ms-clientes	Neutro	CRUD simple con índice único.
ms-inventario	Medio	La reserva de varias líneas exige transacción multidocumento.

5.2 Instancia única en replica set, con base lógica por servicio

Replica set de un nodo: las transacciones multidocumento solo funcionan sobre replica set; un nodo independiente las rechaza. La reserva de stock y el outbox las necesitan.

Instancia única: seis instancias en equipos de estudiantes consumirían memoria sin aportar aislamiento relevante. El patrón de base de datos privada por servicio se cumple con aislamiento lógico: cada servicio tiene su base y un usuario sin permisos sobre las demás. Es una variante reconocida del patrón; su limitación, un único punto de falla físico, se documenta en la sección 8.3.

5.3 Reglas de persistencia derivadas de MongoDB

Regla	Motivo
Campos monetarios como Decimal128	Spring Data MongoDB guarda BigDecimal como texto por omisión; las sumas de ms-reportes fallarían o darían resultados erróneos.
Índices declarados explícitamente	La creación automática está desactivada por omisión en Spring Boot 3; sin declararlos no se aplica la unicidad de correo ni de RUC.
Identificadores ObjectId	No existe autoincremento. Sustituye la convención numérica de la versión anterior.
Folio secuencial en ventas	Un ObjectId de 24 caracteres no es aceptable en un ticket. Se genera con una colección de contadores e incremento atómico.

5.4 Comunicación asíncrona con RabbitMQ y Spring AMQP

Toda comunicación entre servicios de dominio ocurre por eventos a través del broker. El tráfico entre el cliente, el Gateway y los servicios sigue siendo HTTP: una caja registradora necesita respuesta a su petición, y esa comunicación no es entre servicios.

Criterio	RabbitMQ	Kafka
Modelo	Colas y exchanges; mensaje consumido y descartado	Log distribuido reproducible
Adecuación	Eventos de dominio de bajo volumen	Streaming de alto volumen
Operación	Un contenedor con consola incluida	Mayor: particiones, grupos, KRaft

Kafka resuelve problemas que este sistema no tiene. Se usa Spring AMQP directamente, sin la capa de Spring Cloud Stream: para un equipo que aprende, ver exchange, cola y routing key de forma explícita es más formativo y más fácil de depurar.

5.5 Coreografía, no orquestación

Cada servicio publica lo que le ocurrió y reacciona a lo que le interesa. No existe orquestador. Con un único flujo transaccional complejo, un orquestador sería un componente adicional sin justificación.

5.6 Réplica local en ms-ventas y copia histórica

ms-ventas ya no puede consultar a ms-inventario para obtener nombre y precio. Mantiene una réplica mínima del catálogo (identificador, nombre, precio, estado) y de clientes (identificador, nombre), alimentada por eventos. Con ella valida y congela en la venta el nombre y el precio del producto y el nombre del cliente al momento de la operación.

La copia histórica es el modelado correcto del dominio con independencia de la arquitectura: un comprobante reimpresso debe mostrar lo que el cliente compró. La réplica **no incluye existencias**: la autoridad sobre el stock es exclusivamente ms-inventario.

5.7 Proyecciones en ms-reportes

ms-reportes deja de consultar a otros servicios. Construye proyecciones propias de ventas, catálogo y contrapartes a partir de eventos, y genera documentos y agregaciones leyendo solo su base. Ventajas: sin dependencias en tiempo de ejecución y documentos generables aunque otro servicio esté caído. Costo: consistencia eventual.

La configuración de empresa permanece en ms-reportes: sus únicos consumidores son su pantalla de administración y el encabezado de los documentos que este servicio genera.

5.8 Outbox transaccional en ventas e inventario

Si un servicio persiste un dato y luego publica el evento, una caída entre ambas operaciones pierde el evento. En ms-ventas eso deja una venta pendiente para siempre; en ms-inventario, stock descontado sin confirmación. Se aplica el patrón outbox: el evento se escribe en una colección de salida dentro de la misma transacción que el dato, y un publicador la vacía hacia el broker.

Se limita a ventas e inventario, el único flujo donde perder un evento produce inconsistencia de dinero y existencias. En los demás servicios la publicación es directa y la limitación se documenta.

5.9 Validación local del token en el Gateway

El Gateway verifica la firma del token con la clave compartida por variable de entorno, sin consultar a ms-auth. Elimina una llamada síncrona por petición. Eureka conserva su función para el enrutamiento del Gateway; los servicios ya no se descubren entre sí porque no se llaman.

5.10 Decisiones que se mantienen de la versión anterior

Decisión	Justificación resumida
Cientes y proveedores en un solo servicio	Estructura casi idéntica; separarlos produce dos servicios anémicos.
Generación de PDF centralizada en ms-reportes	La utilidad actual es cohesiva; un solo servicio conoce iText.
Facturación excluida	La tabla de facturas aporta un solo campo nuevo. El folio, ahora necesario, es un campo de la venta.

5.11 Sustitución de JavaFX por un cliente web en Vue 3

El equipo aprobó sustituir el cliente de escritorio JavaFX por un SPA en Vue 3. Es sustitución total: no hay cliente dual. Se documenta aquí con el mismo nivel de detalle que las demás decisiones porque introduce dos conceptos que un cliente de escritorio no necesitaba: solicitudes entre orígenes distintos y una estrategia de almacenamiento del token en un entorno donde cualquier script de la página puede intentar leerlo.

Decisión	Justificación
Sustitución total, no cliente dual	Mantener dos clientes activos duplicaría el trabajo de la épica de cliente sin beneficio proporcional en un calendario de 10 semanas.
El contrato de la API no cambia	El cliente sigue siendo un consumidor HTTP más del Gateway; ms-auth y el resto de los servicios no se modifican por este cambio.
Token en memoria (tienda de estado), no en localStorage ni sessionStorage	Ambos son legibles por cualquier script que se ejecute en la página, lo que amplía la superficie de un ataque de scripting entre sitios. El costo es que recargar la página cierra la sesión.
Se descarta el token en cookie httpOnly	Requeriría que ms-auth y el Gateway emitan y validen cookies en lugar de un token portador, lo que reabría trabajo ya cerrado en H-010 y H-014. Se prioriza no generar retrabajo en el backend por un cambio de cliente.

Decisión	Justificación
CORS configurado en el Gateway	Único componente nuevo exigido por tener el cliente en el navegador; se resuelve con una lista de orígenes permitidos por variable de entorno.
Servido mediante contenedor nginx dedicado	Separa el contenido estático del enrutamiento de API; el Gateway sigue siendo el único punto de entrada para las llamadas a los servicios.

Sobre las condiciones originales de esta decisión

La evaluación se había condicionado a un mockup pulido construido y a una estimación en puntos del costo de la sustitución. El equipo aprobó el cambio por consenso, sin que llegara a construirse un prototipo funcional; la estimación en puntos se produjo junto con esta actualización de los documentos. El Product Owner asumió esa omisión como parte de su margen de decisión.

6. Requisitos funcionales por servicio

6.1 ms-auth

ID	Requisito
RF-1.1	Autenticar por correo y contraseña, devolviendo un token firmado.
RF-1.2	Almacenar contraseñas con BCrypt, factor de trabajo 12.
RF-1.3	Validar la política de complejidad en toda contraseña nueva.
RF-1.4	Administrar usuarios: alta, consulta, modificación y baja.
RF-1.5	Asignar a cada usuario el rol ADMIN o VENDEDOR.
RF-1.6	Emitir tokens con un contrato de datos documentado, verificable por el Gateway sin consultar al servicio.
RF-1.7	Rechazar correos duplicados.

6.2 ms-inventario

ID	Requisito
RF-2.1	Administrar productos: alta, consulta, modificación y baja lógica.
RF-2.2	Excluir de los listados los productos inactivos.
RF-2.3	Publicar eventos de alta, modificación y baja de productos.
RF-2.4	Ante una venta creada, reservar las existencias de todas sus líneas de forma atómica.
RF-2.5	Rechazar la reserva si alguna existencia quedaría negativa, indicando el producto y el motivo.
RF-2.6	Publicar un aviso cuando una existencia cae bajo su umbral.
RF-2.7	Procesar cada evento recibido una sola vez, aunque se reciba varias veces.

6.3 ms-ventas

ID	Requisito
RF-3.1	Registrar una venta con una o más líneas, respondiendo su aceptación en estado pendiente.
RF-3.2	Validar productos y cliente contra la réplica local.
RF-3.3	Calcular el total en el servidor, ignorando cualquier total enviado por el cliente.
RF-3.4	Congelar en la venta el nombre y precio de cada producto y el nombre del cliente.
RF-3.5	Mantener la réplica local de catálogo y clientes a partir de eventos.
RF-3.6	Resolver la venta como confirmada o cancelada según la respuesta de inventario, y publicar el resultado.
RF-3.7	Asignar a cada venta un folio secuencial legible.
RF-3.8	Consultar una venta por identificador, incluyendo su estado y sus líneas.
RF-3.9	Listar ventas con filtro por fechas y ordenamiento por campo autorizado.
RF-3.10	Listar las ventas que permanecen pendientes, para detección de inconsistencias.

6.4 ms-clientes

ID	Requisito
RF-4.1	Administrar clientes: alta, consulta, modificación y baja.
RF-4.2	Administrar proveedores: alta, consulta, modificación y baja.
RF-4.3	Validar el formato del correo.
RF-4.4	Rechazar proveedores con RUC duplicado.
RF-4.5	Registrar automáticamente la fecha de alta.
RF-4.6	Publicar eventos de alta y modificación de clientes y proveedores.

6.5 ms-reportes

ID	Requisito
RF-5.1	Administrar la configuración de empresa.
RF-5.2	Mantener proyecciones locales de ventas, catálogo y contrapartes a partir de eventos.
RF-5.3	Generar el comprobante en PDF de una venta confirmada.
RF-5.4	Generar el historial de ventas en PDF por rango de fechas.
RF-5.5	Generar los listados de clientes y de productos en PDF.
RF-5.6	Incluir los datos de la empresa como encabezado en todo documento.
RF-5.7	Entregar datos agregados de ventas por periodo para la gráfica.
RF-5.8	Informar explícitamente cuando un documento aún no está disponible por consistencia eventual.

6.6 ms-notificaciones

ID	Requisito
RF-6.1	Consumir y registrar los eventos de operación recibidos.
RF-6.2	Generar una alerta cuando una existencia cae bajo su umbral.
RF-6.3	Consultar notificaciones con filtro por estado de lectura.
RF-6.4	Marcar una notificación como leída.

7. Requisitos no funcionales

ID	Categoría	Requisito
RNF-01	Arquitectura	Todo servicio se registra en Eureka al arrancar.
RNF-02	Arquitectura	Ningún servicio es accesible externamente salvo a través del Gateway.
RNF-03	Arquitectura	Cada servicio posee su base lógica y un usuario sin permisos sobre las demás.
RNF-04	Seguridad	Autenticación sin estado mediante token firmado, validado localmente en el Gateway.
RNF-05	Seguridad	Contraseñas con BCrypt, factor de trabajo 12.
RNF-06	Seguridad	Ninguna credencial ni clave de firma en el código ni en el repositorio.
RNF-07	Seguridad	La autorización por rol se verifica en el servicio.
RNF-08	Datos	Valores monetarios como Decimal128 en persistencia y decimal exacto en los contratos.
RNF-09	Mensajería	La comunicación entre servicios de dominio es exclusivamente por eventos a través del broker.

ID	Categoría	Requisito
RNF-10	Datos	Índices y datos de prueba versionados y ejecutables desde cero.
RNF-11	Despliegue	El sistema completo se levanta con una sola instrucción de orquestación.
RNF-12	Despliegue	Toda la configuración se externaliza por variables de entorno.
RNF-13	Observabilidad	Registro estructurado con nivel configurable; cada evento registra su identificador.
RNF-14	Calidad	Al menos una prueba automatizada por endpoint y por consumidor principal.
RNF-15	Calidad	El flujo completo de venta se verifica extremo a extremo sobre el sistema levantado desde cero.
RNF-16	Mensajería	Todo consumidor es idempotente: procesar dos veces un evento no produce efecto adicional.
RNF-17	Mensajería	Los mensajes que fallan tras los reintentos configurados terminan en una cola de fallidos.
RNF-18	Mensajería	ms-ventas y ms-inventario publican sus eventos mediante outbox transaccional.
RNF-19	Mensajería	Todo evento incluye identificador único, tipo, versión de esquema, fecha y contenido.

Sobre RNF-16

RabbitMQ entrega cada mensaje al menos una vez, no exactamente una vez. Sin idempotencia, un evento de venta reentregado descuenta el stock dos veces. Es el defecto más probable y más difícil de detectar de toda la arquitectura, porque solo aparece bajo fallos.

8. Delimitación del alcance

8.1 Dentro del alcance

Categoría	Elementos
Arquitectura	Seis servicios. Gateway, Eureka, broker y base de datos.
Mensajería	Eventos de dominio, coreografía, saga de venta, outbox en ventas e inventario, idempotencia, cola de fallidos.
Seguridad	Token validado en el Gateway. Autorización por rol en cada servicio.
Datos	Modelo documental por servicio. Decimal128. Índices declarados. Réplicas y proyecciones.
Funcionalidad	Paridad funcional con MonPointV2 en los dominios implementados.
Despliegue	Imagen por servicio; orquestación con Compose; configuración externalizada.
Cliente	Cliente web en Vue 3 que sustituye a JavaFX, con interfaz rediseñada siguiendo patrones de diseño de aplicaciones web actuales, e incluido el seguimiento de estado de la venta.

Reversa explícita de una decisión previa

La versión 2.0 de este documento excluía el rediseño de la interfaz gráfica, conservando vistas, estilos y animaciones de MonPointV2. La sustitución de JavaFX por Vue 3 vuelve esa exclusión inaplicable: no existe forma de reutilizar vistas FXML en un SPA. El rediseño entra al alcance como consecuencia directa del cambio de tecnología de cliente, no como una decisión de diseño adicional aprobada por separado.

8.2 Fuera del alcance

Elemento	Justificación
Orquestación de contenedores	Excluida; decisión del Product Owner si hubiera margen.
Orquestador central de sagas	Coreografía suficiente para un único flujo transaccional.
Kafka y procesamiento de flujos	Resuelve problemas de volumen que el sistema no tiene.
Reconstrucción de estado desde eventos	Los eventos notifican cambios; no son la fuente de verdad.
Comunicación en tiempo real con el cliente	El cliente consulta el estado de la venta; no se usan WebSocket ni SSE.
Outbox en clientes, reportes y notificaciones	Limitado al flujo de dinero y existencias. Ver 8.3.
Pruebas automatizadas de integración entre servicios	Verificación extremo a extremo manual en H-094 y H-095.
Facturación fiscal y registro de facturas	Fuera de escala; el folio es un campo de la venta.
Exportación a hoja de cálculo	Dependencia nunca utilizada.
Instalador de escritorio	El despliegue objetivo es por contenedores.
Persistencia local de sesión y rol OTRO	Sustituida por token; rol sin semántica.
Migración de datos reales de MonPointV2	Se usan datos de prueba cargables.

8.3 Limitaciones aceptadas

- **Consistencia eventual en ms-reportes:** un comprobante solicitado en el mismo instante de la confirmación puede no estar disponible; el cliente reintenta.
- **Publicación directa fuera de ventas e inventario:** un cambio de cliente puede no reflejarse en réplicas si el servicio cae tras persistir. Impacto: nombre desactualizado en ventas futuras.
- **Orden de eventos no garantizado bajo reintentos:** réplicas y proyecciones descartan un evento más antiguo que el estado almacenado.
- **Clave de firma compartida:** quien la posea puede emitir tokens. Sin revocación: un token es válido hasta su expiración.
- **Instancia única de MongoDB:** aislamiento lógico, no físico; punto único de falla.
- **Sin reintentos HTTP ni cortocircuito** entre Gateway y servicios.

9. Comunicación entre servicios: eventos y saga

9.1 Catálogo de eventos

Evento	Productor	Consumidores
ProductoCreado · ProductoActualizado · ProductoDadoDeBaja	ms-inventario	ms-ventas (réplica) · ms-reportes (proyección)
ClienteCreado · ClienteActualizado	ms-clientes	ms-ventas (réplica) · ms-reportes
ProveedorCreado · ProveedorActualizado	ms-clientes	ms-reportes
VentaCreada	ms-ventas	ms-inventario
StockReservado · StockRechazado	ms-inventario	ms-ventas
StockBajo	ms-inventario	ms-notificaciones
VentaConfirmada · VentaCancelada	ms-ventas	ms-reportes · ms-notificaciones

Topología: un exchange de tipo topic por dominio (inventario.eventos, clientes.eventos, ventas.eventos), una cola por consumidor y una cola de fallidos por cola. El esquema detallado de cada evento es el entregable H-008 y queda congelado en la semana 4.

9.2 Estructura común de un evento

```
{
  "eventoId": "UUID unico, clave de idempotencia",
  "tipo": "VentaCreada",
  "version": 1,
  "fecha": "2026-10-12T14:32:05Z",
  "origen": "ms-ventas",
  "datos": { ...contenido especifico del evento... }
}
```

9.3 Saga de registro de venta

1. Cliente -> POST /ventas (via Gateway, HTTP)
2. ms-ventas valida contra su replica local, congela nombre y precio, persiste la venta PENDIENTE con folio y escribe VentaCreada en su outbox dentro de la misma transaccion
-> responde 202 Accepted {id, folio, estado: PENDIENTE}
3. Publicador vacia el outbox -> RabbitMQ (exchange ventas.eventos)
4. ms-inventario consume VentaCreada (idempotente)
-> transaccion: reserva todas las lineas o ninguna
-> outbox: StockReservado o StockRechazado {producto, motivo}
-> si alguna existencia cae bajo umbral: StockBajo
5. ms-ventas consume la respuesta (idempotente)
-> CONFIRMADA + VentaConfirmada | CANCELADA + VentaCancelada
6. ms-reportes consume VentaConfirmada -> actualiza su proyeccion
ms-notificaciones consume StockBajo -> genera alerta
7. Cliente consulta GET /ventas/{id} hasta estado final

Consecuencia para la interfaz

La caja no recibe «venta registrada» en la respuesta del POST, sino «venta en proceso». El cliente web consulta el estado hasta que es final, muestra el estado intermedio y maneja el rechazo posterior por existencia. En condiciones normales la resolución toma menos de un segundo.

9.4 Garantías de mensajería

Garantía	Mecanismo
Ningún evento se pierde entre escritura y publicación (ventas, inventario)	Outbox transaccional
Ningún evento produce efecto dos veces	Registro de identificadores procesados por consumidor
Un fallo transitorio no descarta el mensaje	Reintento con límite
Un mensaje que falla siempre no bloquea la cola	Cola de mensajes fallidos
Un evento antiguo no sobrescribe uno reciente	Comparación de fecha en réplicas y proyecciones

10. Trazabilidad y criterios de verificación

REQUISITO (Documento 02) -> HISTORIA (Documento 03) -> INCIDENCIA -> CONFIRMACIONES

Un requisito está verificado cuando todas sus historias cumplen los criterios de terminado del Documento 01 y la funcionalidad es demostrable sobre el sistema en ejecución.

10.1 Cambios de comportamiento deliberados

Comportamiento actual	Comportamiento nuevo
Sesión persistida en archivo cifrado	Sesión sostenida por token con vigencia definida
La venta se confirma al instante	La venta se acepta pendiente y se confirma de forma asíncrona
El ticket identifica la venta por su número interno	El ticket muestra un folio secuencial
El ticket reimpresso muestra el nombre actual del producto	Muestra el nombre vigente en la fecha de la venta
Totales en coma flotante	Totales en aritmética decimal exacta
Control de acceso ocultando controles	Control de acceso verificado en el servidor
Migración automática de contraseñas en texto plano	Toda contraseña se almacena cifrada desde el alta

Documento 02, versión 3.1. Añade la decisión de sustituir JavaFX por Vue 3 (sección 5.11) y corrige la exclusión de rediseño de interfaz, ya inaplicable. Alcance cerrado en lo demás. Ninguna incorporación, sustitución o ampliación procede sin autorización del Product Owner. Las 58 historias que implementan estos requisitos se detallan en el Documento 03.